

Automated Lead Generation Pipeline (n8n + SerpAPI + Google Sheets)

An n8n workflow that turns **a niche + a location** into a structured, deduplicated list of business leads, with websites and contact emails, stored in Google Sheets.

Example input: **Niche:** Dentists · **Location:** Karachi · **Country:** Pakistan

Output: rows in a Google Sheet with business name, website, phone, address, rating, reviews and extracted emails.

Demo video: <add link here>

Table of contents

- 1 What it does
- 2 Tech stack
- 3 Architecture
- 4 How a run works (step by step)
- 5 Node-by-node reference
- 6 Filtering rules
- 7 Email enrichment
- 8 State management and the stop mechanism
- 9 Google Sheet structure
- 10 Setup guide
- 11 How to use
- 12 Limits and known constraints
- 13 Troubleshooting and lessons learned
- 14 Roadmap
- 15 Responsible use

What it does

Manually building a lead list means searching Google Maps, opening each website, copying details and maintaining a spreadsheet. This workflow automates that loop:

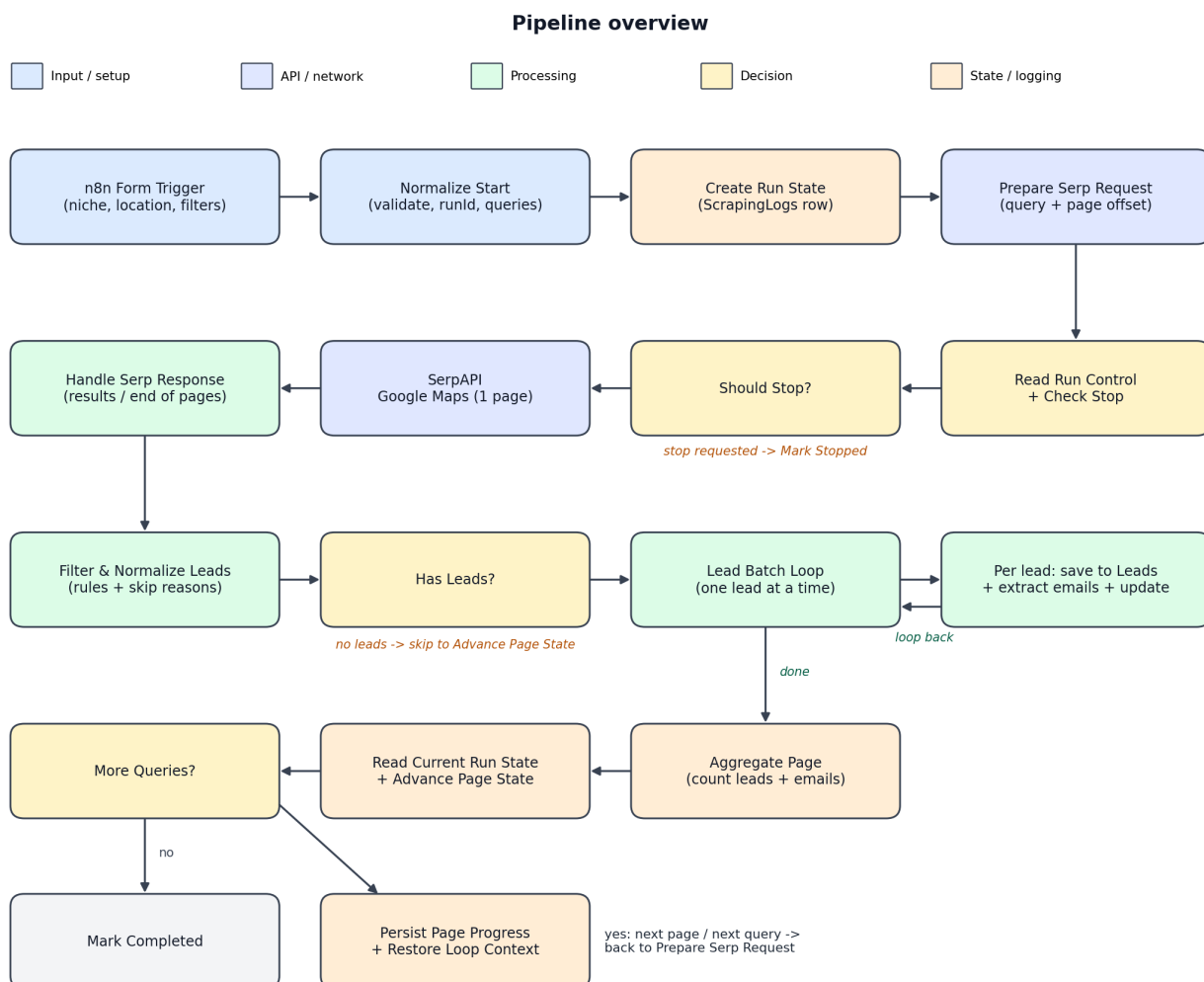
Stage	What happens
Discover	Queries Google Maps through SerpAPI and paginates page by page
Filter	Drops weak or irrelevant results (rating, review count, no website, social-media-only pages)
Enrich	Visits each business website and extracts contact emails
Store	Saves leads to Google Sheets, deduplicated by website
Track	Logs every run (pages, leads, skipped, errors) so it can be monitored and debugged
Control	A run can be stopped mid-scraper from the log sheet

Tech stack

Tool	Role
n8n	Workflow automation (orchestration, logic, loops)
SerpAPI (google_maps engine)	Google Maps results with pagination
Google Sheets	Lead storage and run-state/log storage
n8n Form Trigger	Input form (niche, location, filters)

Tool	Role
JavaScript (Code nodes)	Filtering, normalization, email extraction, state handling

Architecture



The workflow has two nested loops:

- **Outer loop (pages):** one SerpAPI request per page (20 results per page), then the next page, then the next search variant.
- **Inner loop (leads):** every lead on the page is saved and enriched one at a time.

How a run works (step by step)

- 1 **Form submit.** The user enters niche, location, country, minimum rating, minimum reviews, website required (Yes/No), email enrichment (Yes/No) and search mode (focused/broad).
- 2 **Normalize Start.** Form labels are mapped to internal names, validated, and a unique **runId** is generated. The country is converted to a SerpAPI **gl** code (for example Pakistan becomes **pk**). Search query variants are built:
 - *Focused:* "<niche> in <location> <country>", "<niche> <location> <country>"
 - *Broad:* three phrasing variants
- 3 **Create Run State.** A row is written to the **ScrapingLogs** sheet with status **running**.
- 4 **Prepare Serp Request.** Picks the current query variant and page offset (**start**).

- 5 **Read Run Control and Check Stop.** Before every page the workflow re-reads its own log row. If `stop_requested` is `true`, it branches to **Mark Stopped**.
- 6 **SerpAPI Google Maps.** Fetches one page of results.
- 7 **Handle Serp Response.** Extracts `local_results`. SerpAPI returns an error message when pagination runs out of results; this is treated as a normal end of the query, not a failure.
- 8 **Filter and Normalize Leads.** Applies the filtering rules, builds clean lead objects and records *why* each result was skipped.
- 9 **Has Leads?** If nothing survived filtering, the page is skipped and pagination continues.
- 10 **Lead Batch Loop.** For each lead:
 - Save to the `Leads` sheet (with or without a website)
 - If enrichment is on and the lead has a website: fetch the HTML, extract emails, update the row
- 11 **Aggregate Page.** After the loop finishes, all processed leads are collapsed into one item (lead count and email count for the page). This keeps later nodes from running once per lead.
- 12 **Read Current Run State and Advance Page State.** Updates counters and decides the next move:
 - Page returned results: `start += 20`
 - No results or error: move to the next query variant
- 13 **More Queries?** If variants remain, persist progress and go back to step 4. Otherwise **Mark Completed**.

Node-by-node reference

Node	Type	Purpose
Form Trigger	n8n Form Trigger	Collects run inputs
Normalize Start	Code	Maps and validates inputs, builds <code>runId</code> , query variants, initial counters
Create Run State	Google Sheets (append or update)	Creates the run row in <code>ScrapingLogs</code>
Restore Start Context	Code	Restores the full run context after the Sheets node replaces <code>\$json</code>
Prepare Serp Request	Code	Chooses the current query and offset
Read Run Control	Google Sheets (read)	Reads the run row to check for a stop request
Check Stop	Code	Sets <code>shouldStop</code> from <code>stop_requested</code>
Should Stop?	IF	Routes to stop or continue
SerpAPI Google Maps	HTTP Request	Calls <code>serpapi.com/search</code> with <code>engine=google_maps</code>
Handle Serp Response	Code	Extracts results and classifies API errors
Filter and Normalize Leads	Code	Filtering, URL normalization, skip-reason counters
Has Leads?	IF	Skips empty pages safely
Expand Lead Batch	Code	Turns the lead array into individual items
Lead Batch Loop	Split In Batches	Processes one lead at a time
Has Website To Save	IF	Chooses the save path
Save Lead With Website	Google Sheets (append or update)	Upsert keyed on <code>website</code>
Save Lead Without Website	Google Sheets (append)	Used when websites are not required
Enrich Emails?	IF	Honors the form setting
Has Website For Email	IF	Only fetches sites that exist
Get Web HTML	HTTP Request	Downloads the homepage (browser User-Agent, tolerant of bad SSL, errors do not stop the run)
Extract and Filter Emails	Code	Finds and cleans email addresses

Node	Type	Purpose
Update Lead Email	Google Sheets (append or update)	Writes emails and <code>is_enriched</code>
Update Lead Counter	Code	Emits per-lead email count for aggregation
Aggregate Page	Code	Collapses the page's leads into one summary item
Read Current Run State	Google Sheets (read)	Reads the run row, filtered by <code>run_id</code>
Advance Page State	Code	Computes counters, next offset, next query
More Queries?	IF	Continue or finish
Persist Page Progress	Google Sheets (append or update)	Writes page-level progress and a readable status message
Restore Loop Context	Code	Restores context before the next page
Mark Completed / Mark Stopped	Google Sheets	Final status updates

Filtering rules

Applied in `Filter and Normalize Leads`:

Rule	Behaviour
Website required	If enabled, businesses without a website are skipped
Social-only pages	Instagram, Facebook, TikTok, LinkedIn, X, YouTube, WhatsApp, Linktree and similar are skipped as "websites"
Minimum rating	Skipped if below the threshold
Minimum reviews	Skipped if below the threshold
Niche relevance	Only used in <i>Broad</i> mode. Matches niche stems against the business name, description and Google category. In <i>Focused</i> mode the Google Maps ranking is trusted
Deduplication	Within a page by <code>place_id</code> / website; across runs by <code>website</code> in the sheet

Every page logs a readable summary, for example:

```
Page 3: 20 found, 14 kept • skipped: no_website 4, reviews 2
```

URLs are normalized to the root domain (`https://example.com`) so the same business is not saved twice.

Email enrichment

For each lead with a website, the workflow downloads the homepage and extracts addresses from both `mailto:` links and plain text (including HTML-encoded @).

Emails are rejected when they:

- belong to placeholder domains (`example.com`, `domain.com`, etc.)
- are generic non-contact addresses (`noreply`, `no-reply`, `webmaster`, `wordpress...`)
- look like file names (`logo@2x.png`)
- have obvious dummy local parts (`test`, `username`, `yourname`, ...)

Multiple emails per business are stored comma-separated. Only the homepage is crawled, so businesses that publish emails on a contact page will not always yield one.

State management and the stop mechanism

All run state lives in the `ScrapingLogs` sheet (one row per run, keyed by `run_id`):

- Counters: `leads_saved`, `pages_processed`, `api_errors`, `emails_found`, `skipped`

- Position: `query_index`, `start`
- Control: `status`, `stop_requested`, `message`
- Timestamps: `started_at`, `updated_at`, `completed_at`

Stopping a run: open `ScrapingLogs`, find the run row and set `stop_requested` to `true`. The workflow checks this before every page and exits cleanly with status `stopped`.

Google Sheet structure

Create one spreadsheet with two tabs.

Tab `ScrapingLogs` (header row):

```
run_id | status | niche | location | country | min_rating | min_reviews | website_required |
enrich_emails | query_index | start | leads_saved | pages_processed | api_errors |
emails_found | skipped | stop_requested | message | started_at | completed_at | updated_at
```

Tab `Leads` (header row):

```
business_name | email | website | description | niche | ratings | reviews | reviews_url |
location | phone | address | data_scrapped | is_enriched | email_sent | email_subject |
email_body | sent_at
```

Tip: format the `phone` column as **Plain text** so numbers like `+92...` are not converted.

The `email_sent`, `email_subject`, `email_body`, `sent_at` columns are reserved for the upcoming outreach stage.

Setup guide

Prerequisites

- An n8n instance (cloud or self-hosted)
- A [SerpAPI](#) account and API key
- A Google account with access to Google Sheets

Steps

- 1 **Create the spreadsheet** with the two tabs and headers above.
- 2 **Import the workflow:** n8n, then *Workflows*, *Import from file*, and select `workflow.json`.
- 3 **Add credentials** in n8n:
 - *SerpAPI* credential (your API key)
 - *Google Sheets OAuth2* credential
- 4 **Point every Google Sheets node** to your own spreadsheet and the correct tab (`ScrapingLogs` or `Leads`).
- 5 **Set cell format** on the lead-saving nodes: *Options*, *Cell Format*, *Let Google Sheets format* (keeps phone numbers as text).
- 6 **Configure the Form Trigger** with these fields:

Label	Type
Niche	Text (required)
Location	Text (required)
Country	Text (default: Pakistan)
Minimum Rating	Number (default 3.5)
Minimum Reviews	Number (default 5)
Website Required	Dropdown: Yes / No
Enrich Emails	Dropdown: Yes / No
Search Mode	Dropdown: focused / broad

The labels must match what **Normalize Start** reads.

7 **Activate** the workflow and open the Form Trigger's **Production URL**.

How to use

- 1 Open the form URL, enter a niche and a city, submit.
- 2 Watch progress in **ScrapingLogs** (the **message** column updates every page).
- 3 Collect results from the **Leads** tab.
- 4 To abort, set **stop_requested = true** on the run row.

Tips

- Start with *Website Required = No* and low thresholds to confirm the pipeline works, then tighten filters.
- Use *Focused* mode for precise niches, *Broad* mode when you want wider coverage.
- Each SerpAPI page consumes API credits, so test with small runs first.

Limits and known constraints

Constraint	Detail
Google Sheets read quota	About 60 reads per minute per user. append or update nodes read the sheet internally. The workflow limits reads (no per-lead progress writes, one aggregated read per page), but very fast runs can still hit the quota. Add a short Wait node in the lead loop if needed
SerpAPI credits	One credit per page request
Result depth	Google Maps pagination typically runs out after several pages per query
Email coverage	Homepage only; many businesses do not publish an email
Websites that block bots	Some sites return errors; the run continues and the lead is saved without an email
Progress UI	Progress is tracked in the sheet, not in a custom web UI

Troubleshooting and lessons learned

Symptom	Cause	Fix
0 leads, no error	A URL-parsing step failed silently, so every business looked like it had no website and was filtered out	Replaced new URL(...) handling with a regex-based normalizer, and added per-page skip-reason logging
Flow stops after the first page	A node downstream of a Sheets node was reading fields that only exist in the original context	Added "Restore Context" Code nodes after Sheets nodes that replace \$json
Stop button never worked	Progress writes kept overwriting stop_requested back to false	Progress nodes no longer write status or stop_requested
Quota exceeded for Read requests	Each append or update performs a read; per-lead writes plus a loop output that triggered the next node once per lead exceeded 60 reads/min	Removed per-lead progress writes and added the Aggregate Page node
Phone numbers corrupted	Sheets interpreted +92... as a formula or number	Plain-text column plus leading apostrophe with <i>Let Google Sheets format</i>
Fake emails like logo@2x.png	Regex matched image file names	Added a file-extension filter
Unexpected token 'const'	Code pasted together with leftover lines or Markdown fences	Replace the whole code box and paste only the code

Main lesson: a pipeline that fails silently is worse than one that crashes loudly. Log *what came in, what was kept, what was dropped, and why*.

Roadmap

- [] AI-personalised outreach emails per lead (LLM node writing `email_subject` and `email_body`)
 - [] Lead scoring from website content
 - [] Contact-page crawling for better email coverage
 - [] Automated, rate-limited email sending with `email_sent` / `sent_at` tracking
 - [] Multi-city batch runs from a list
-

Responsible use

- Respect each website's terms of service and `robots.txt`.
 - Follow applicable regulations for cold outreach (for example CAN-SPAM, GDPR, local anti-spam laws): identify yourself, offer an opt-out and honor it.
 - Use the data for legitimate business purposes only.
-

Repository contents (suggested)

```
.
├── README.md
├── workflow.json           # exported n8n workflow (remove credentials / sheet IDs first)
├── screenshots/
│   ├── workflow-overview.png
│   ├── form.png
│   └── results-sheet.png
└── demo/
    └── demo-video-link.md
```

Before pushing to GitHub: open `workflow.json` and replace your Google Sheet ID/URL and credential IDs with placeholders. Credentials are referenced by ID only (API keys are not stored in the JSON), but your spreadsheet link is.